딥러닝 캐스케이드 웹 공격 탐지: 넘김 판단의 한계와 배포 조건별 속도–정확도 교환의 재평가

Deep-Learning Cascades for Web Attack Detection: Limits of Deferral Decisions and a Re-evaluation of Speed–Accuracy Trade-offs under Deployment Conditions

요 약

웹 요청을 48×48 바이트 이미지로 변환해 얕은 합성곱 신경망(convolutional neural network, CNN)으로 분류하는 방식은 문자 수준 CNN(character-level CNN, char-CNN)보다 Macro-F1이 유의하게 낮았다(0.8949 대 0.9884). 본 논문은 이 음성 결과에서 출발하여, 빠른 1차 모델이 모든 요청을 먼저 처리하고 확신도가 낮은 요청만 무거운 2차 모델로 넘기는 캐스케이드의 속도–정확도 교환을 실트래픽 기반 SR-BH 2020 데이터셋에서 재평가한다. 첫째, 이미지 CNN 1차의 속도 이득은 측정 방식에 의존하였다. 2차 입력을 최대 길이로 채우고 순전파만 재면 1.22배였으나, 전처리를 포함하고 실제 요청 길이로 재면 1차(1.311ms)가 2차(0.393ms)보다 느려 0.27배가 되었으므로 사전 고정 규칙에 따라 해당 속도 이득 주장을 철회한다. 둘째, 확신도 기반 넘김 판단은 이상적 라우터보다 평균 10.5배 많이 넘겼고, 대체 신호·클래스별 임계값·학습형 라우터·특징 공간 거리 점수 모두 사전 채택 기준을 넘지 못했다. 또한 공격 비율이 높은 시험 집합의 넘김률만으로 게이트를 고르면 실제 배포 공격 비율(3.3%)에서는 정상 요청 넘김 때문에 오히려 더 느린 구성을 택하게 됨을 보인다. 셋째, 1·2차를 모두 딥러닝으로 구성한 세 가지 대안을 같은 교차검증 분할과 단일 스레드 CPU에서 비교하였다. char-CNN 내부 조기 종료는 단독 모델 대비 3.5~3.6배 빠르고 Macro-F1 손실이 0.10%p 이내였으나, 그 이득은 넘김 판단이 아니라 결합 학습된 얕은 분기에서 나왔다. char-CNN을 1차로, 미세조정한 DistilBERT를 2차로 둔 캐스케이드는 DistilBERT가 SQL 삽입을 명령 삽입(CmdI)으로 오분류하던 오류를 절반 이하(seed 42에서 192→99건, seed 43에서 252→2건)로 줄이는 특성을 활용한다. 이 구성은 실사용자 정상 트래픽 기준 DistilBERT 단독보다 평균 비용 기준 12.3배(seed 42) 및 10.6배(seed 43) 빨랐고, seed 42에서는 Macro-F1 손실 0.03%p로 사전 기준을 충족하였으나 확인용 seed 43에서는 손실이 0.32%p(허용 0.30%p)로 경계를 넘어 재현되지 않았다. 캐스케이드의 실효성은 전처리를 포함한 지연 정의, 배포 공격 비율, 클래스별 오분류 구조, 분할 seed를 함께 보고할 때에만 판단할 수 있다.

ABSTRACT

Classifying web requests as 48×48 byte images with a shallow convolutional neural network (CNN) yielded significantly lower Macro-F1 than a character-level CNN (char-CNN; 0.8949 vs. 0.9884). Starting from this negative result, we re-evaluate on the real-traffic SR-BH 2020 dataset the speed–accuracy trade-off of cascade detectors, in which a fast first stage screens every request and defers only low-confidence requests to a heavier second stage. First, the apparent speedup of an image-CNN first stage was an artifact of the timing protocol: timing only the forward pass on inputs padded to the maximum length gave 1.22×, whereas timing end-to-end latency on unpadded lengths made the first stage (1.311 ms) slower than the second stage (0.393 ms), yielding 0.27× and leading us to withdraw the claim under a pre-specified rule. Second, confidence-based deferral routed 10.5× as many requests as an oracle router on average, and none of the tested alternatives—temperature-scaled or margin/entropy signals, class-wise thresholds, learned routers, or feature-space distance scores—met pre-specified acceptance criteria. We further show that selecting gates by test-split deferral rates picks configurations that are slower at deployment attack rates (3.3%) due to elevated deferral on benign traffic. Third, we compare three all-deep-learning cascades on the same cross-validation folds and single-thread CPU setup. An internal early-exit char-CNN was 3.5–3.6× faster than the stand-alone char-CNN within 0.10 percentage points of Macro-F1, though this gain stemmed from the jointly trained shallow branch rather than deferral. A char-CNN → DistilBERT cascade leverages DistilBERT's ability to cut SQL-injection-to-command-injection (CmdI) false positives by more than half (192→99 on seed 42; 252→2 on seed 43). On real-user benign traffic, the cascade was 12.3× (seed 42) and 10.6× (seed 43) faster in average cost than DistilBERT alone; it met the pre-specified accuracy

tolerance on seed 42 (−0.03 percentage points) but narrowly missed confirmation on seed 43 (−0.32 points vs. a 0.30-point limit). Cascade gains depend on end-to-end latency definitions, deployment base rates, class-specific error patterns, and split seeds, and should be reported with these conditions made explicit.

Keywords: Web attack detection, Cascade inference, Deferral decision, Early exit, Command injection, Inference latency

I. 서론

웹 애플리케이션 방화벽(web application firewall, WAF)에 기계학습 분류기를 배치할 때는 탐지 정확도뿐 아니라 요청 한 건을 처리하는 지연 시간을 함께 따져야 한다. WAF는 모든 인바운드 HTTP 요청의 처리 경로 위에 놓이므로, 분류기가 수십 밀리초씩 걸리면 정상 서비스의 응답 시간도 그대로 늦어진다. 동시에 보안 분야의 기계학습 평가에서는 실험실 측정 조건과 실제 배포 환경의 괴리가 성능을 과장한다는 점이 반복해 지적되어 왔다[1].

본 연구는 당초 악성코드 바이너리를 회색조 이미지로 변환해 분류하는 기법[2]을 웹 보안 도메인으로 옮기면, HTTP 요청을 바이트 이미지로 바꾼 CNN이 텍스트 기반 신경망과 비슷하거나 더 나은 분류 성능을 내는지(Q1)를 확인하는 데서 출발하였다. 교차검증 결과 답은 "아니오"였다(5.1절). 바이트를 2차원 격자로 접는 과정에서 인접 문맥이 끊기면서 문자 수준 합성곱 신경망(character-level CNN, char-CNN)보다 Macro-F1이 약 9%p 낮게 나타났다. 사후에 연구 질문을 바꾸면 결과를 보고 가설을 고치는 셈이 되므로, 이 음성 결과를 그대로 보고하고 후속 설계의 출발점으로 삼는다.

다만 고정 길이(2,304바이트) 입력의 순전파 시간만 놓고 보면 얕은 이미지 CNN은 char-CNN보다 연산량이 적었다. 여기서 두 번째 질문이 파생되었다. 가벼운 1차 모델이 모든 요청을 먼저 판정하고 확신도가 임계값 τ 미만인 요청만 무거운 2차 모델로 넘기는 캐스케이드(cascade)를 구성하면, 2차 단독 수준의 정확도를 더 낮은 평균 지연으로 얻을 수 있는가(Q2). 이전 단계에서 공개 공격 페이로드 모음과 GPU 배치 추론으로 재었을 때는 같은 정확도에서 5.07배 빨랐다. 그러나 실제 WAF는 요청이 한 건씩 도착하는 단일 스트림 환경에 가깝고 트래픽의 대다수가 정상 요청이다. 게다가 전처리 시간과 가변 길이 입력을 반영했을 때에도 같은 속도 이득이 유지되는지, 그리고 1차 확신도만 보고 넘김 여부를 정하는 게이트가 실제로2차 모델이 고칠 수 있는 오답을 제대로 골라내는지(Q3)[6]는 별도로 검증해야 한다.

나아가 본 연구는 제안하는 탐지 구성을 수동 특징 공학(n-gram 빈도 사전 등)에 의존하지 않고 원시 바이트열이나 서브워드 토큰을 종단간(end-to-end)으로 학습하는 딥러닝 모델들로 한정한다(고전 기계학습인 TF-IDF+로지스틱 회귀는 대조 기준선으로만 둔다). 실험 과정에서 가장 까다로운 병목은 공격과 정상을 가르는 이진 차단보다, 국소 구분자(;, |, ' 등)를 공유하는 SQL 삽입(SQL injection, SQLi)과 명령 삽입(command injection, CmdI) 사이의 공격 종류 오탐이었다. 경량 문자 모델(char-CNN)은 평균 0.4~0.6ms 안에 요청을 처리하지만 짧은 합성곱 창 때문에 SQLi를 CmdI로 오분류하는 경향이 남고, 사전학습 트랜스포머인 DistilBERT[12]는 전역 어텐션으로 이 혼동을 크게 줄이는 대신 단일 스레드 CPU에서 약 45ms가 걸려 단독 인라인 배치가 어렵다. 이 속도–정확도 간극을 캐스케이드와 조기 종료로 어떻게 메울 수 있는지가 후반부 실험의 핵심이다.

실트래픽 기반 다중 공격 데이터셋인 SR-BH 2020[3] 위에서 수행한 본 논문의 기여는 세 가지이다.

- 전처리 포함 지연과 배포 조건에 따른 재평가. 이미지 CNN 1차 캐스케이드의 속도 이득이 지연 정의(RGB 이미지 변환 포함 여부, 2차 입력의 패딩 제거)와 배포 공격 비율(π = 3.3%)에 따라 1.22배에서 0.27배로 역전됨을 보이고, 사전 고정 규칙에 따라 요청 단위 CPU 처리에서의 속도 이득 주장을 철회한다.
- 넘김 판단의 구조적 한계와 평가 지표 규명. 기본 확신도 게이트는 오라클 라우터보다 평균 10.5배 많은 요청을 넘겼으며, 온도 보정·마진·엔트로피, 클래스별 임계값, 내부 표현 라우터, 특징 공간 거리 점수 4종 모두 사전 채택 기준을 넘지 못했다. 특히 공격 비율이 74.8%인 시험 집합의 전체 넘김률로만 게이트를 고르면 정상 요청 넘김률(r_N)이 높아져 배포 환경에서 오히려 더 느린 게이트를 선택하게 됨을 실측으로 보이고, 실사용자 정상 층의 r_N과 π 재가중 평균 비용 배수를 함께 보고하는 평가 절차를 제시한다.

- 딥러닝 캐스케이드**(DL-A·DL-B·DL-C)**의 속도**,** **CmdI** 오탐 감소**,** 정확도**–**속도 교환 분석**.** 1·2차를 모두 딥러닝으로 구성한 세 대안을 동일 교차검증 분할과 동일 CPU에서 비교하고, 선행 연구의 DistilBERT 탐지기[5]를 같은 조건에서 재구현해 기준선으로 삼는다. char-CNN 내부 조기 종료(DL-C)는 결합 학습된 얕은 분기만으로 3.5~3.6배 가속을 달성했고, char-CNN → DistilBERT 캐스케이드(DL-A)는 DistilBERT의 CmdI 오탐 억제 능력을 일부 흡수하며 평균 지연을 10.6~12.3배 단축하되 분할 seed에 따라 정확도 허용폭(0.30%p) 충족 여부가 갈리는 경계를 정량적으로 보고한다.

결과에 맞추어 판정 기준을 사후 변경하는 일(HARKing)을 막기 위해, 모든 확인 실험의 채택 조건은 학습 및 측정 이전에 버전 관리 기록에 먼저 고정하였다(사전 판정 문서 커밋 2a456d7, 4b7843b, f5a002a, 7a1f725, 00c52ce, f37fd70). 사전 기준에 없던 추가 분석은 본문에서 사후 분석 또는 병기 값으로 명확히 구분한다.

II. 관련 연구

2.1 이미지 기반 보안 탐지

Nataraj 등[2]은 악성코드 실행 파일의 바이트열을 고정 폭 회색조 이미지로 재배치한 뒤 질감 특징으로 악성코드 계열을 분류하였다. 이후 소스코드 문맥을 이미지 채널로 인코딩해 소프트웨어 취약점을 탐지하거나[7], 어셈블리 명령어의 구문·통계 특성을 RGB 세 채널에 나누어 담는 방식[8]으로 확장되었다. 본 연구의 1차 이미지 모델은 이 계열을 웹 요청 페이로드에 옮겨 바이트 값, 문자 범주, 지역 엔트로피를 48×48×3 격자에 배치한 것이며, 5.1절과 5.2절에서 왜 이 2차원 표현이 가변 길이 웹 요청의 실시간 처리에는 불리한지를 검증한다.

2.2 텍스트 기반 웹 공격 탐지와 SR-BH 2020

웹 공격 탐지 연구의 주류는 HTTP 요청 문자열을 1차원 시퀀스로 직접 다룬다. Tadhani 등[9]은 1차원 CNN과 LSTM을 결합해 SQLi와 교차 사이트 스크립팅(cross-site scripting, XSS)을 분류하였고, Zhang 등[13]의 문자 수준 CNN(char-CNN)은 별도의 토크나이저 없이 원시 문자·바이트 시퀀스에서 지역 n-gram 패턴을 추출하는 경량 기준선으로 널리 쓰인다.

Sureda Riera 등[3]이 공개한 SR-BH 2020은 실제 운영 웹 서버 앞단의 ModSecurity와 OWASP 핵심 규칙 집합(core rule set, CRS) 로그를 검수해 CAPEC 공격 분류 체계를 붙인 다중 클래스 데이터셋이다. 15년 전 합성 트래픽인 CSIC 2010과 달리 실제 스캐너와 브라우저 요청의 URI, 본문, 헤더를 모두 담고 있어 최근 연구의 주 실험 무대가 되었다. 같은 데이터셋 위에서 Osama 등[5]은 중복 제거와 라벨 정제를 거친 WAMM 프레임워크를 제안하며 XGBoost와 DistilBERT[12]의 분류 성능 및 추론 지연을 보고하였다. Luo 등[10]은 BERT 기반 이중 채널 구조(WADBERT)를, Bitussi와 Doriguzzi-Corin[11]은 SR-BH의 오라벨을 교정한 판본에서 설명 가능한 이상 탐지(X-WAD)를 제안하였다. 본 연구는 Osama 등[5]이 보고한 딥러닝 최고 모델인 DistilBERT를 동일 분할·동일 CPU 조건에서 재구현해 캐스케이드의 직접 비교 기준선으로 삼는다.

2.3 캐스케이드 추론과 넘김 판단

Wang 등[14]은 별도로 학습된 모델들을 최대 소프트맥스 확률(maximum softmax probability, MSP)[17] 임계값으로 잇는 것만으로도 단일 대형 모델보다 수 배 빠른 파레토 경계를 얻을 수 있음을 보였다. 웹 보안에서도 Tasdemir 등[4]이 가벼운 1차 분류기와 BERT 2차 분류기를 결합해 SQLi 단일 공격 탐지에서 BERT 단독 대비 약 20배의 추론 가속을 보고하였고, 2단계 스태킹 앙상블 WAF[15]나 제로데이 탐지용 경량 캐스케이드[16]도 제안되었다. 그러나 선행 연구들은 2차 기준 모델의 크기, 배치 크기, 하드웨어가 제각각이어서 보고된 가속 배수를 직접 대조하기 어렵다.

어느 요청을 2차로 넘길지 결정하는 문제는 선택적 분류(selective classification)[18] 및 넘김 학습(learning to defer)[19][20]과 맞닿아 있다. Kag 등[21]은 엣지 모델과 라우터를 결합 학습하였고, Enomoto와 Eda[22]는 일반적인 온도 보정[32]이 캐스케이드의 비용–정확도 교환에는 오히려 해로울 수 있음을 보였다. 특히 Jitkrittum 등[6]은 이론적 최적 넘김 규칙이 1차 확신도가 아니라 "2차 정답 확률과 1차 정답 확률의 차이"에 의해 결정되므로, 2차가 특정 하위 분포에만 강하거나 데이터에 라벨 잡음이 섞여 있을 때 1차 확신도 게이트가 구조적으로 실패함을 증명하였다. 단일 신경망 내부에 보조 분류기를 달아 쉬운 입력을 일찍 내보내는 조기 종료(early exit)[23]는 두 모델 간 전처리 중복을 없앤 대안이다. 한편

분포 밖(out-of-distribution, OOD) 탐지에서 쓰이는 마할라노비스 거리 등 특징 공간 거리 점수[24]–[27]도 넘김 점수의 후보가 될 수 있으나, Jaeger 등[28]은 실제 오분류 탐지에서는 단순 MSP가 대다수 거리 점수보다 낫다는 실증 결과를 보고한 바 있다.

2.4 추론 지연 측정과 기저율 함정

적응형 추론 구조는 입력을 변조해 조기 종료를 방해하고 연산 비용을 폭증시키는 지연 유발 공격(slowdown attack)[29]에 노출된다. 또한 신경망의 추론 시간은 요청을 1건씩 처리하는 단일 스트림 조건과 대량 배치 조건에서 연산 병목이 전혀 다르므로, MLPerf Inference[30]는 시나리오를 엄격히 분리해 측정한다. 마지막으로 침입 탐지에서는 실제 환경의 공격 기저율(base rate)이 낮기 때문에 작은 정상 오탐이나 정상 넘김도 전체 경보와 비용을 지배한다는 Axelsson[31]의 기저율 문제가 캐스케이드 지연 평가에도 그대로 작동한다.

III. 방법

3.1 입력 표현과 바이트 이미지 변환

각 HTTP 요청에서 URI와 본문(body)을 이어 붙인 원시 바이트열을 $b = (b_0, b_1, ..., b_{L-1})$이라 한다. 이미지 기반 모델은 앞 2,304바이트를 잘라(짧으면 0으로 패딩) 48×48 2차원 격자에 행 우선으로 배치한다.

$$x[r, c] = b[48r + c], 0 \le r, c < 48 \quad (1)$$

단일 회색조 대신 바이트의 구문적 성질을 함께 주려고 RGB 3채널 구성을 쓴다. R 채널은 정규화한 원시 바이트 값, G 채널은 ASCII 문자 범주(영숫자, 공백, SQL/셸 구분자, 제어문자 등), B 채널은 주변 5바이트 창의 지역 섀넌 엔트로피이다(Fig. 1). 다만 SR-BH 2020의 웹 요청은 길이 중앙값이 96바이트(p95 233바이트, p99 349바이트)에 불과하여, 48×48 격자에서 실제로 채워지는 영역은 위쪽 2~3줄이 대부분이고 나머지 90% 이상은 0 패딩으로 남는다.

반면 문자 수준 모델(char-CNN 계열)은 같은 바이트열 b를 2차원으로 접지 않고 실제 길이 L 그대로의 1차원 시퀀스로 읽는다(합성곱 최대 커널 크기인 7바이트보다 짧은 경우만 7바이트까지 채움). 트랜스포머 모델(DistilBERT)은 원문 문자열을 WordPiece 토크나이저(소문자화 포함)로 분절해 최대 512 토큰까지 입력받는다.



Fig. 1. Examples of 48×48 RGB byte images constructed from SR-BH 2020 web requests (R: raw byte value, G: character class, B: local byte entropy).

3.2 비교 모델과 딥러닝 캐스케이드 구성

Table 1은 본 논문에서 비교하는 모델들의 구조와 학습 설정이다. 모든 딥러닝 모델은 동일한 5-fold 교차검증 분할 위에서 클래스 빈도 역수 가중 교차 엔트로피(class-weighted cross-entropy)로 학습하였다(단, 5.2절의 고정 분할 지연 기준선에만 기존 언더샘플링 체크포인트를 함께 대조하였다).

- **RGB CNN**: 3×3 합성곱 3단(채널 32→64→128, 배치 정규화·ReLU·2×2 최대 풀링)과 전역 평균 풀링(global average pooling, GAP)으로 구성된 93,988 파라미터의 얕은 2차원 CNN이다.
- **1-D CNN**: RGB CNN과 똑같은 3채널(바이트·문자 범주·지역 엔트로피) 수열을 48×48로 접지 않고 3×2,304 1차원 수열로 두어 커널 크기 3의 1차원 합성곱과 전역 최대 풀링으로 읽는 대조 모델이다(1차 재설계 실험용).
- **char-CNN**[13]: 각 바이트(패딩 포함 257개 기호)를 64차원 벡터로 임베딩한 뒤, 커널 크기 3·5·7의 1차원 합성곱 분기(각 128채널)와 시간축 최대 풀링을 병렬 수행해 384차원 특징을 뽑고 선형 분류기에 넣는다(141,252 파라미터). 이미지 1차 캐스케이드의 2차 모델이자, 딥러닝 전용 실험의 기본 기준선 및 DL-A의 1차 모델로 쓰인다.
- **char-CNN-tiny**: DL-B의 1차 모델로, 임베딩을 32차원으로 줄이고 커널 3 분기 하나(64채널)만 남긴 14,692 파라미터의 초경량 문자 CNN이다.

- **char-CNN-EE**: DL-C(내부 조기 종료)를 위해 char-CNN의 커널 3 분기 풀링 출력(128차원) 뒤에 보조 선형 분류기(128→4)를 추가한 구조이다(141,768 파라미터). 학습 시 주 헤드 손실과 보조 헤드 손실을 $L = L_{main} + 0.3\, L_{aux}$로 결합해 동시에 최적화한다. 추론 시에는 바이트 임베딩과 커널 3 분기만 먼저 계산해 보조 헤드의 확신도가 임계값 이상이면 즉시 종료하고, 미달할 때만 나머지 커널 5·7 분기를 마저 계산한다. 보조 헤드 경로와 동일한 크기(임베딩 64, 커널 3 하나, 128채널, 41,668 파라미터)를 처음부터 단독 학습한 대조 모델을 **char-CNN-k3**라 부른다.
- **DistilBERT**[12]: 사전학습된 `distilbert-base-uncased`(66.96M 파라미터) 위에 4클래스 분류 헤드를 얹어 학습률 2×10^{-5}, 배치 크기 16으로 3 에폭 미세조정하였다. 이는 같은 SR-BH 2020 데이터셋에서 딥러닝 최고 성능을 보고한 Osama 등(WAMM)[5]의 구성을 동일 분할·동일 CPU에서 직접 비교하기 위해 재구현한 것이다. 원 논문은 최대 길이를 128 토큰으로 두었으나, 본 데이터셋의 학습+검증 풀에서 128 토큰 제한은 전체 요청의 6.9%, 특히 길이가 긴 CodeInjection의 24.7%와 CmdI의 26.5%를 잘라내어 비교 상대에게 불리하게 작용한다. 따라서 주 실험(E10·E11)에서는 잘림이 0%인 512 토큰(요청 토큰 수 중앙값 54, p99 189)을 사용해 참조 모델의 상한 성능을 보존하였다.

Table 1. Models compared in 5-fold cross-validation on SR-BH 2020

Model	Role in experiments	Input representation	Parameters	Training setup
RGB CNN	Q1; image first stage	48×48×3 byte grid	93,988	class-weighted CE, lr 1e-3
1-D CNN	E2 first-stage redesign	3×2,304 1-D channels	46,660	class-weighted CE, lr 1e-3
char-CNN	2nd stage; 1st stage of DL-A; reference	raw bytes (unpadded)	141,252	class-weighted CE, lr 1e-3
char-CNN-tiny	1st stage of DL-B	raw bytes (unpadded)	14,692	class-weighted CE, lr 1e-3
char-CNN-k3	single-branch control for DL-C	raw bytes (unpadded)	41,668	class-weighted CE, lr 1e-3
char-CNN-EE	DL-C (internal early exit)	raw bytes (unpadded)	141,768	joint CE (main + 0.3 aux), lr 1e-3
DistilBERT	2nd stage of DL-A; WAMM[5] reference	up to 512 WordPiece tokens	66.96M	class-weighted CE, Adam, lr 2e-5, 3 ep.
TF-IDF + LR	classical linear baseline only	char_wb 2–4-grams (20k)	80,004	class-weighted L2 logistic reg.

Notes: All neural models use early stopping on internal validation Macro-F1 (patience 5, up to 30 epochs except DistilBERT with 3 epochs). WAMM[5] originally used 13 classes, 128 tokens, AdamW, and 8 vCPUs.

3.3 캐스케이드 추론과 넘김 판단(게이트) 변형

1차 모델 h_1이 요청 x에 대해 출력한 소프트맥스 확률 분포를 $p_1(k|x)$, 기본 확신도 점수를 최대 소프트맥스 확률 $s(x) = \max_k p_1(k|x)$라 하자. 임계값 τ에 따른 2단 캐스케이드의 최종 예측 $\hat{y}(x)$는 다음과 같다.

$$\hat{y}(x) = h_1(x) \text{ if } s(x) \geq \tau, \text{ otherwise } h_2(x) \quad (2)$$

단일 임계값 기본 게이트(**B0**)는 각 교차검증 fold의 내부 검증 집합에서 넘김 예산에 대응하는 확신도 분위수로 τ를 정한 뒤 시험 집합에 1회 적용한다. 운영점은 검증 집합의 Macro-F1이 2차 단독 모델 검증 Macro-F1의 0.5%p 이내를 유지하는 지점 중 배포 넘김률(3.4절 식 5)이 가장 작은 임계값으로 고른다. 내부 조기 종료(DL-C)는 식 (2)에서 h_1을 커널 3 보조 헤드로, h_2를 커널 3·5·7 전체를 거친 주 헤드로 둔 것과 같다.

본 연구는 B0의 과잉 넘김 원인을 분리하기 위해 네 갈래의 게이트 변형을 체계적으로 대조하였다.

1. 확신도 신호 교체(**G1**): 사후 온도 보정[32]을 적용한 MSP(msp_T), 상위 두 클래스 확률의 차이인 마진 $s_{margin}(x) = p_1((1)|x) - p_1((2)|x)$, 음의 섀넌 엔트로피 $s_{ent}(x) = \Sigma_k p_1(k|x) \log p_1(k|x)$ 세 가지로 $s(x)$를 대체한다.
2. 예측 클래스별 임계값(**G2**): 클래스마다 확신도 분포와 오분류 비용이 다른 점을 반영해, 1차 예측 클래스 $\hat{c}_1(x) = \text{argmax}_k p_1(k|x)$별로 서로 다른 임계값 τ_c를 둔다. defer $x \Leftrightarrow s(x) < \tau_{\hat{c}_1(x)}$, $\hat{c}_1(x) = \text{argmax}_k p_1(k|x)$ (3)

임계값 벡터 (τ_1, ..., τ_4)는 내부 검증 집합에서 전체 넘김 예산을 0.5%p씩 늘릴 때 단위 넘김당 Macro-F1 상승폭이 가장 큰 클래스에 예산을 배정하는 탐욕적(greedy) 경로 탐색으로 결정한다.

3. 학습형 라우터**(G3)**: Jitkrittum 등[6]의 정식화를 따라 "1차는 틀리고 2차는 맞히는 요청인가"($I[h_1(x) \neq y \wedge h_2(x) = y]$)를 직접 예측하는 로지스틱 회귀 라우터를 내부 검증 집합에서 학습한다. 1차 확률과 요청 길이·엔트로피 통계만 넣은 경량판(G3-lite)과 1차 GAP 128차원 내부 표현까지 결합한 확장판(G3-full)을 모두 측정한다.
4. 특징 공간 거리 점수**(G6)**: 확률 출력 p_1이 아니라 1차 GAP 128차원 특징 공간 z(x)에서 학습 데이터 분포와의 거리를 재는 비모수·모수 점수 네 가지를 비교한다. 폴드 내부 학습 집합에서 추정한 클래스별 평균과 Ledoit–Wolf 수축 공분산을 쓰는 마할라노비스 거리(maha)[24], 단일 배경 가우시안 거리를 뺀 상대 마할라노비스 거리(rmaha)[25], L2 정규화 특징 은행(클래스당 2,000건)에서의 k=50 최근접 이웃 거리(knn)[26], 그리고 예측 클래스 최근접 거리 대 타 클래스 최근접 거리의 비를 쓰는 Trust Score(trust)[27]이다.

3.4 지연 정의, 배포 공격 비율 재가중, 평균 비용 배수

요청 한 건의 1차 처리 시간을 t_1, 2차 처리 시간을 t_2, 넘김 확률을 r이라 하면, 캐스케이드의 요청당 기대 지연 t와 비교 기준 모델(지연 t_{ref}) 대비 속도 배수 S는 다음과 같다.

$$t = t_1 + r \cdot t_2,\ S = t_{ref} / t \tag{4}$$

여기서 이미지 1차 캐스케이드와 DL-B의 기준 모델은 char-CNN 단독($t_{ref} = t_2$)이다. 내부 조기 종료(DL-C)는 앞단(인코딩+임베딩+커널 3+보조 헤드) 시간을 t_{front}, 뒷단(커널 5·7+주 분류기) 시간을 t_{rest}라 할 때 $t = t_{front} + r \cdot t_{rest}$이며 기준은 $t_{ref} = t_2$이다. 반면 char-CNN → DistilBERT 캐스케이드(DL-A)는 모든 요청이 char-CNN($t_1 = t_2$)을 거치고 일부만 DistilBERT(t_R)로 넘어가므로 기대 지연이 $t = t_2 + r \cdot t_R$이고, 주 비교 기준은 재구현 DistilBERT 단독($t_{ref} = t_R$)이다.

중요한 점은 넘김률이 정상 요청(r_N)과 공격 요청(r_A)에서 크게 다르다는 것이다. 배포 환경의 공격 기저율이 π일 때 실제 기대 넘김률 E_π는 다음과 같이 가중 평균된다.

$$E_\pi = \pi \cdot r_A + (1 - \pi) \cdot r_N \tag{5}$$

SR-BH 2020 시험 집합은 공격 요청이 74.8%를 차지하지만 실제 운영 웹 서버의 공격 기저율은 훨씬 낮다. 본 논문은 선행 캐스케이드 연구인 Tasdemir 등[4]의 설정과 맞추어 $\boldsymbol{\pi = 3.3\%}$를 기본 배포 공격 비율로 삼는다. 또한 정상 요청 넘김률 r_N은 검증 분포 전체(스캐너 탐색 요청이 96% 이상)로 잰 값과, 교차검증 시험 폴드 내 실사용자 브라우저 정상 요청(550건)만으로 잰 값을 분리해 병기하고 주 판정은 실사용자 기준으로 수행한다.

교차검증의 5개 폴드에서 폴드별 속도 배수 S_f를 단순 산술평균하면, 넘김이 많아 지연이 급증한 느린 폴드의 비용이 가려져 배포 속도를 과대평가하게 된다(예: 한 폴드가 80배, 다른 폴드가 3배일 때 산술평균은 41.5배이지만 평균 지연 기준으로는 5.8배). 따라서 본 논문은 다섯 폴드의 평균 비용으로 계산한 합산 속도 배수 $\bar{S}$를 주 지표로 사용한다.

$$\bar{S} = t_{ref} / \left((1/5) \Sigma_f t_f \right) \tag{6}$$

식 (6)은 폴드별 속도 배수 S_f의 조화평균(harmonic mean)과 정확히 일치한다.

3.5 오라클 대비 과잉 넘김 배율

게이트 자체의 선별 효율을 모델 지연과 독립적으로 재기 위해 과잉 넘김 배율(excess deferral ratio) ρ를 정의한다. 이상적 라우터(오라클)는 1차가 틀린 요청만 정확히 골라 넘기며 그때의 넘김률을 e_o라 한다. 평가 대상 게이트가 2차 단독 모델의 시험 Macro-F1에 도달하기 위해 필요한 최소 넘김률을 e_g라 할 때, 과잉 넘김 배율은 $\rho = e_g / e_o$이다. $\rho = 1$이면 게이트가 오라클과 동등한 효율을 낸 것이고, 값이 클수록 불필요한 정상·정답 요청을 많이 넘긴 것이다.

IV. 실험 환경

4.1 데이터셋과 정상 트래픽 층 분리

SR-BH 2020[3] 원본 907,815행의 다중 CAPEC 라벨을 정상(Normal), SQL 삽입(SQLi), 코드 삽입(CodeInjection), 명령 삽입(CmdI)의 4클래스로 매핑하였다. 이어서 정규표현식 시그니처 감사로 정상 라벨에 공격 구문이 섞인 의심 행 30,280건, 동일 입력 문자열에 서로 다른 라벨이 붙은 충돌 행, 그리고 완전 중복 행을 제거해 최종 265,939행을 구성하였다(Table 2). 여기서 CodeInjection 클래스는 클라이언트 측 XSS가 아니라 서버 측 PHP 코드 실행 탐침이 주를 이룬다. 또한 원본 데이터의 라벨은 HTTP 헤더까지 보고 부여되었으나 모델 입력은 URI와 본문(F2)만 사용한다. 쿠키 필드를 추가한 입력(F3)은 쿠키 값의 내용을 마스킹한 절제 입력(F3M)과의 성능 차이가 통계적으로 유의하지 않아(Holm 보정 $p = 0.17$), 불필요한 식별자 과적합을 피하기 위해 URI+본문 입력을 유지하였다.

Table 2. SR-BH 2020 dataset composition after deduplication and label audit (rows per split)

Split	Normal	SQLi	CodeInj.	CmdI	Total
Train	46,864	128,502	6,905	3,886	186,157
Validation	10,042	27,537	1,480	832	39,891
Test (sealed)	10,042	27,536	1,480	833	39,891

데이터 분석 과정에서 드러난 중요한 특성은 정상(Normal) 요청의 출처 편향이다. 중복 제거 후 정상 요청의 97.8%가 취약점 스캐너(Nikto, OpenVAS 등)의 User-Agent를 달고 자동 크롤링 중 생성된 짧은 요청이었고, 일반 브라우저 User-Agent로만 들어온 순수 실사용자 정상 요청은 전체에서 645건(학습+검증 풀에서 550건)뿐이었다. 실제로 char-CNN 기준 실사용자 정상 층의 오탐률(false positive rate, FPR)은 2.73%로 스캐너 정상 층(0.36%)의 약 7.6배에 달했다. 따라서 스캐너 위주의 전체 정상 지표를 실운영 성능으로 해석하면 오탐과 넘김 비용을 크게 과소평가하게 되므로, 모든 게이트 및 캐스케이드 평가에서 실사용자 정상 층을 별도로 분리해 보고한다.

4.2 평가 절차와 사전 고정 판정 기준

다중 클래스 불균형(CmdI가 전체의 2.1%)을 반영해 주 분류 지표는 클래스별 F1의 비가중 평균인 **Macro-F1**을 사용한다. 설계 선택과 주 가설 판정은 봉인된 시험 집합(39,891행)을 빼고 학습+검증 풀 **226,048**행을 층화 5-fold로 나눈 교차검증에서 수행한다(RGB CNN, char-CNN 계열, DistilBERT 모두 동일한 226,048행 풀과 동일한 폴드 인덱스를 공유함을 라벨 지문 해시로 검증하였다). 각 폴드의 내부 학습 부분(80%)을 다시 9:1로 갈라 조기 종료 및 게이트 임계값(τ) 선택용 내부 검증 집합으로 쓴다. 폴드 간 성능 차이의 유의성 검정에는 학습 집합 중복에 따른 분산 과소추정을 교정하는 Nadeau–Bengio 보정 대응 t-검정[33]과 Holm 다중 비교 보정을 적용한다. 분할 및 초기화 seed는 42를 기본 판정에, 43(게이트 확인은 43~45)을 사전 등록된 재현성 확인에 사용한다.

추론 지연은 단일 컴퓨터(AMD Ryzen 7 9800X3D, RAM 32GB, NVIDIA RTX 4080 SUPER, Windows 11, PyTorch 2.6.0)에서 다른 연산 프로세스가 없는 상태로 측정하였다. 인라인 WAF의 단일 스트림 처리[30]를 반영하여 **CPU** 배치 **1**·단일 스레드**(1 thread)** 조건에서 원문 문자열 입력부터 전처리(이미지 변환·바이트 인코딩·토큰화)와 신경망 순전파를 거쳐 최종 예측이 나오기까지의 종단간(end-to-end) 시간을 잰다. 검증 집합에서 추출한 2,000개 요청을 20회 워밍업 후 30회 반복 측정하여 중앙값(p50)을 대푯값으로 쓰며, 실행 시점마다 CPU 클럭 표류로 절대값이 4~6% 흔들릴 수 있으므로 비교 대상 모델들은 반드시 한 실행 안에서 요청마다 순서를 번갈아 측정해 그 실행 내부의 비율로만 속도 배수를 계산한다.

교수 피드백에 따른 "기준 모델 대비 개선(그래프에서 위)" 판정은 측정 전에 다음 세 조건을 모두 충족하는 것으로 고정하였다(실사용자 정상 분포, $\pi = 3.3\%$ 기준): ① 합산 평균 비용 배수 $\bar{S} \geq 1.2$배, ② 5개 폴드 모두 $S_f > 1.0$배, ③ 기준 단독 모델 대비 교차검증 Macro-F1 평균 손실이 0.30%p 이하($\Delta F1 \geq -0.30\%p$)일 것.

V. 실험 결과

5.1 이미지 단독 탐지의 한계(Q1)

5-fold 교차검증(seed 42)에서 48×48×3 RGB CNN의 단독 Macro-F1은 0.8949±0.0140이었던 반면, 같은 바이트열을 읽는 char-CNN은 0.9884±0.0037을 기록해 이미지 모델이 9.35%p 낮았다(보정 t-검정 $p < 0.001$). 이전 데이터셋(CSIC 2010 정상 결합 트랙)에서도 0.9804 대 0.9954로 방향이 같았다.

원인을 분리하기 위해 동일한 3채널(원시 바이트·문자 범주·지역 엔트로피) 특징을 48×48 2차원 격자로 접지 않고 1차원 수열로 그대로 합성곱한 **1-D CNN**(E2)을 같은 폴드에서 학습한 결과, Macro-F1이 0.9341로 RGB CNN보다 **+3.92%p** 상승하였다(Holm 보정 $p = 0.009$). 즉 이미지 모델의 정확도를 깎아먹은 주원인은 3채널 인코딩 정보가 아니라, 1차원 문자열을 48바이트마다 줄바꿈해 2차원 격자로 접으면서 문맥을 끊고 세로로 무관한 위치의 바이트들을 합성곱 커널에 묶어 넣은 **2**차원 재배치 자체였다. 따라서 가변 길이 웹 요청에서 행 우선 바이트 이미지와 얕은 2차원 CNN은 단독 탐지기로서 텍스트 신경망을 대체하지 못한다.

5.2 이미지 1차 캐스케이드: 측정 조건에 따른 속도 이득의 소멸(Q2)

패딩·순전파 기준의 착시. 초기 실험에서는 2차 char-CNN 입력도 이미지와 똑같이 2,304바이트로 0 패딩하고 텐서 변환 이후의 순전파 시간만 측정하였다(이하 채움 기준). 이 조건에서는 고정 분할 시험 집합 기준 1차 RGB CNN(언더샘플링 학습) → 2차 char-CNN 캐스케이드가 운영점(넘김 50.3%)에서 1.22배, 배포 공격 비율(π = 3.3%)에서 0.99배였다. 1차 학습을 클래스 가중치 방식으로 바꾸면 정상 요청에 대한 1차 오탐과 불확실성이 줄어 시험 집합 배수가 1.63배, $\pi = 3.3\%$ 배수가 1.42배(단독 실행 기준 1.67배·1.45배)로 올라갔다. 그러나 이 학습 방식 변경 효과를 5-fold 교차검증으로 다시 검증하면 검증 분포에서 +27%(1.05→1.34배, $p = 0.11$), 실사용자 정상 분포에서는 +4.9%(0.97→1.01배, $p = 0.70$)에 그쳤다.

종단간 공정 지연 기준에서의 역전. 전처리를 포함하고 2차 char-CNN이 패딩 없이 요청의 실제 바이트 길이(중앙값 96바이트)만 처리하도록 공정 지연(E1)을 측정하자 결과가 완전히 뒤집혔다(Table 3). 단일 스레드 CPU에서 1차 RGB CNN의 종단간 지연은 **1.311ms**(RGB 이미지 변환 전처리 0.407ms + 순전파 0.879ms)인 반면, 실제 길이를 읽는 2차 char-CNN의 종단간 지연은 **0.393ms**(바이트 인코딩 0.009ms + 순전파 0.384ms)에 불과했다. 2차 입력을 2,304바이트로 강제 패딩하면 2차 지연이 2.990ms로 약 7.6배 부풀려졌던 것이다.

Table 3. End-to-end single-request latency on CPU (batch 1, 1 thread, median p50 in ms) and deployment speedup over the stand-alone char-CNN at $\pi = 3.3\%$ (5-fold CV, seed 42)

First-stage model	Stage-1 latency t_1 (ms)	Upper bound t_2/t_1	Stand-alone Macro-F1	Deferral E_π (val mix)	S (val mix)	S (real-user benign)
RGB CNN (48×48×3)	1.311	0.300×	0.8949	43.7%	0.265×	0.249×
1-D CNN (3×2,304)	1.284	0.306×	0.9341	29.8%	0.281×	0.265×
TF-IDF + LR (classical ref.)	0.295	1.331×	0.9406	5.3%	1.244×	0.858×

Notes: Second stage is char-CNN with end-to-end unpadded latency $t_2 = 0.393$ ms (under the old 2,304-byte padded forward-only protocol, t_2 was 2.990 ms). Operating points maintain validation Macro-F1 within 0.5 pp of stand-alone char-CNN (0.983–0.984 test Macro-F1 across all rows).

Table 3에서 보듯 이미지 1차(1.311ms)가 2차(0.393ms)보다 3.3배 느리므로, 넘김률을 0%로 만들어도 속도 배수 상한이 $t_2/t_1 = 0.30$배를 넘을 수 없고 실제 캐스케이드 배수는 **0.27**배(실사용자 0.25배)에 머문다. 1-D CNN으로 1차 정확도를 높여 배포 넘김률 E_π를 43.7%에서 29.8%로 줄여도, 3채널 전처리(0.407ms) 자체가 이미 char-CNN 전체 추론 시간(0.393ms)보다 길기 때문에 배수는 0.28배로 사실상 변하지 않는다. 이에 따라 사전 판정 규칙(E1-a)대로 "이미지 1차 캐스케이드의 CPU 단일 요청 속도 이득" 주장을 철회한다.

한편 고전 선형 기준선인 TF-IDF+로지스틱 회귀 1차(0.295ms)는 이미지 1차보다 4.7배 빨라 검증 분포에서 1.24배를 냈으나, 실사용자 정상

요청에서는 넘김이 늘어 다시 0.86배로 떨어졌다. 1·2차 예측 오류의 독립성을 나타내는 Yule의 Q 통계량[34]은 RGB CNN 1차(0.954)가 TF-IDF 1차(0.996)보다 낮아 표현이 다를수록 오류가 덜 겹쳤지만, 정작 운영점 Macro-F1은 모든 1차에서 0.983~0.984로 동일했다. 즉 **1·2**차 간 표현 이질성은 속도 이득의 필요조건이 아니었으며**(E3)**, 캐스케이드가 성립하기 위한 제1조건은 전처리를 포함한 1차 지연 t_1이 2차 지연 t_2보다 압도적으로 작은 것이다.

5.3 넘김 판단(게이트)의 구조적 한계(Q3)

1차 RGB CNN(클래스 가중치)과 2차 char-CNN 위에서 측정한 게이트 변형들의 5-fold 교차검증 결과는 Table 4와 같다. 기본 단일 임계값 게이트(B0)가 2차 단독 Macro-F1에 도달하려면 오라클 라우터보다 평균 **10.53**배(폴드 범위 6.1~17.7배) 많은 요청을 넘겨야 했다.

Table 4. Deferral-gate variants evaluated in 5-fold cross-validation (class-weighted RGB CNN first stage → char-CNN second stage, seed 42)

Deferral variant	Primary metric	Change vs. B0	Corrected p	Pre-specified verdict
Alternative signals (msp_T / margin / entropy)	Excess deferral ρ	−4.3% to −4.7%	0.96 (Holm)	Rejected (need ≥ 20% cut)
Class-wise thresholds (G2)	Excess deferral ρ	−18.7%	0.42	Rejected ($p \geq 0.05$)
Multi-seed frontier check of G2 (seeds 43, 44, 45)	Frontier / op.-point S	3/3 above; +15.7% at s43	0.17–0.41	Curve above; op. point unconfirmed
Learned router on 128-D features (G3-full)	Excess deferral ρ	−5.9%	0.69	Rejected (need ≥ 20% cut)
Feature-space distances (maha, rmaha, knn, trust)	Deployment deferral E_π	+46% to +128%	≤ 0.046 (worse)	Rejected (significantly worse)

Notes: B0 mean excess deferral ratio ρ = 10.53. Distance scores were also slower or less accurate at failure detection (AUROC 0.20–0.87 vs. MSP 0.938).

네 갈래의 게이트 개선 시도는 모두 사전 채택 기준(20% 이상 감소 및 $p < 0.05$)을 통과하지 못했다. 첫째, 온도 보정 MSP·마진·엔트로피(G1)는 B0와의 요청 순위 Spearman 상관계수가 0.997 이상이어서 사실상 똑같은 순서로 요청을 넘겼고 과잉 넘김을 4.3~4.7% 줄이는 데 그쳤다. 둘째, 1차의 128차원 내부 표현을 입력받은 학습형 라우터(G3-full)도 교차검증 과잉 넘김 배율을 5.9%밖에 줄이지 못했다($p = 0.69$). 셋째, "확신도 수치 자체가 잘못된 것 아닌가"라는 가설을 검증한 특징 공간 거리 점수 4종(G6)은 오히려 B0보다 배포 넘김률 E_π를 **46~128%** 증가시켰다. 1차 오답 탐지 AUROC를 재어 보면 단순 MSP가 0.938인 데 비해 마할라노비스·k-NN·Trust 점수는 0.20~0.87로 현저히 낮았다. 이는 Jaeger 등[28]의 지적대로, 분류기가 틀리는 웹 요청이 특징 공간 바깥의 이상치(OOD)가 아니라 클래스 결정 경계가 겹치는 분포 한가운데에 놓여 있기 때문이다. 즉 어떤 점수로 불확실성을 재는가는 병목이 아니었다. 넷째, 예측 클래스별 임계값(G2)은 과잉 넘김 배율을 18.7% 줄였고, 새로 학습한 확인용 seed 43·44·45 세 번 모두에서 정확도–속도 경계 곡선이 B0보다 위에 놓였으나(1차 규칙 3/3 충족), 폴드 5개 규모의 낮은 검정력(20% 감소 검출 검정력 7~10%)으로 인해 보정 p값은 0.17~0.41에 머물렀다. 검증 집합에서 고른 단일 운영점의 속도 이득 또한 판정 기준인 seed 43에서 +15.7%로 사전 기준(+20%)에 미달하였다.

그렇다면 왜 모든 게이트가 오라클보다 10배나 많이 넘겨야만 2차 단독 성능에 닿는가? 교차검증 풀에서 **1**차가 확신도 **0.99** 이상으로 강하게 확신하며 틀렸고 **2**차**(char-CNN)**는 라벨과 일치한 요청 **110**건을 추출해 대조군(1·2차 모두 정답인 고확신 요청 110건)과 함께 블라인드 원문 감사를 수행한 결과(대형 언어모델 단일 판정자), 대상 집단의 **70.9%(78/110**건**)**가 원문과 데이터셋 라벨이 맞지 않는 오라벨·의심 행으로 판정되었다(대조군은 31.8%, 오즈비 5.2). 예컨대 `|cat /etc/passwd#`나 백틱 명령 주입 구문이 SQLi 또는 Normal 라벨로 붙어 있거나, 헤더에만 공격 흔적이 있고 URI·본문은 완전히 비어 있는 요청들이었다. 즉 1차가 고확신으로 "틀린" 꼬리 구간의 상당수는 모델의 판단 실패가 아니라 데이터셋의 라벨 잡음이며, 2차

모델이 그 잡음 패턴까지 암기해 맞힌 점수를 캐스케이드가 따라잡으려다 보니 임계값을 0.99 근처까지 끌어올려 대량의 정상 요청을 넘기게 되는 것이다.

5.4 배포 기저율과 실사용자 분포에 따른 게이트 평가의 함정

게이트 실험에서 드러난 또 하나의 방법론적 결과는 시험 집합의 전체 넘김률만으로 게이트를 평가하면 배포 환경의 속도를 정반대로 오판한다는 점이다(Table 5). 고정 분할 시험 집합(언더샘플링 1차)에서 사전 기준을 통과한 G2(시험 넘김 −8.8%p)와 G3-full(−6.7%p)은 공격 요청의 넘김률(r_A)을 대폭 낮춘 대신 정상 요청의 넘김률(r_N)을 오히려 67.2%에서 71.0%(G2)와 78.1%(G3-full)로 높였다. 공격이 74.8%인 시험 집합에서는 전체 넘김률이 줄어 빨라진 것처럼 보이지만, 공격이 3.3%인 배포 비율에서는 E_π가 r_N에 수렴하므로 기준을 통과한 G2(0.99배)와 G3-full(0.91배)이 기본 게이트 B0(1.02배)보다 실제로는 더 느렸다.

Table 5. Reversal between test-split deferral rate and deployment speedup on the fixed split (undersampled RGB CNN first stage → char-CNN second stage, padded latency protocol)

Gate	Test deferral (π = 74.8%)	Test Macro-F1	Benign deferral r_N	Attack deferral r_A	S at test mix	S at deployment (π = 3.3%)
B0 (single τ)	48.25%	0.9803	67.2%	41.9%	1.255×	1.022×
G2 (class-wise τ)	39.47%	0.9795	71.0%	28.9%	1.411×	0.990×
G3-lite (stat router)	47.59%	0.9809	98.3%	30.5%	1.241×	0.772×
G3-full (128-D router)	41.56%	0.9806	78.1%	29.3%	1.385×	0.909×

더 나아가 교차검증(클래스 가중치 1차)에서 정상 요청을 출처별로 가르면, 실사용자 브라우저 정상 요청**(550**건**)**의 넘김률 $\mathbf{r_N}$은 **69.3%**로 스캐너가 섞인 검증 분포 전체의 r_N(44.4%)보다 **+24.9%p** 높았다(보정 $p = 1.7\times10^{-5}$, 사전 판정 기준 H-G4 충족). 스캐너의 정형화된 탐색 요청보다 실제 사용자의 일반 웹 요청이 훨씬 다양해 1차 모델의 확신도가 낮게 형성되기 때문이다. 이 실사용자 r_N을 대입하면 채움 기준으로도 B0의 배포 배수는 1.34배에서 1.01배로 무너진다. 따라서 이후 5.5절의 모든 딥러닝 캐스케이드 판정은 실사용자 정상 층의 $\mathbf{r_N}$과 $\boldsymbol{\pi}$ = **3.3%**를 반영한 평균 비용 배수**(**식 **6)**를 기준으로 삼는다.

5.5 딥러닝 캐스케이드: 속도, 정확도–속도 교환, 그리고 작동 조건

5.2절에서 확인했듯 캐스케이드가 속도 이득을 내려면 전처리를 포함한 1차 지연이 2차보다 훨씬 작아야 한다. 바이트 시퀀스를 직접 읽는 char-CNN은 이미 단일 스레드 CPU에서 약 0.35~0.56ms로 동작하면서 Macro-F1 0.9884를 내므로, 딥러닝만으로 이보다 나은 속도–정확도 교환을 얻는 경로는 구조적으로 세 가지뿐이다(E7~E11). 첫째, char-CNN 앞에 더 작은 초경량 문자 모델을 두는 것**(DL-B: char-CNN-tiny → char-CNN**). 둘째, 별도 1차 모델을 두지 않고 char-CNN 내부의 가장 싼 커널 3 분기 뒤에서 조기 종료하는 것**(DL-C: char-CNN-EE**). 셋째, 빠르고 정확한 char-CNN을 1차로 내리고 훨씬 무거운 사전학습 트랜스포머를 2차로 두는 고전적 캐스케이드 구도**(DL-A: char-CNN → DistilBERT**)이다.

본 학습에 앞서 수행한 층별 지연 분해 관문 실험(E7)에서 char-CNN 추론 시간의 **91.0%**가 세 합성곱 층**(**커널 **3·5·7)**에서 소비되고 바이트 인코딩과 프레임워크 호출 고정비는 8.5%에 불과함이 확인되었다. 덕분에 커널을 줄이거나 앞 분기만 실행하면 단일 배치 CPU에서도 실제 지연이 비례해서 줄어들어, 넘김 5.3% 가정 시 DL-B는 최대 7.24배, DL-C는 최대 3.03배의 이론적 상한을 가졌다. 또한 DL-A의 2차 후보인 재구현 DistilBERT 단독 모델은 5-fold 교차검증에서 Macro-F1 0.9914(seed 42)를 기록해 char-CNN 단독(0.9884)을 넘어 관문(E10-1)을 통과하였다(차이 +0.30%p, 보정 $p = 0.57$). Table 6과 Fig. 2는 세 딥러닝 구성의 본 측정 및 확인 실험 결과를 종합한 것이다.

Table 6. Speed–accuracy trade-off of all-deep-learning cascades vs. stand-alone references (5-fold CV, CPU batch 1, 1 thread, end-to-end unpadded latency, real-user benign traffic at π = 3.3%)

Configuration	Reference model (F1)	Seed	Gate	Macro-F1	ΔF1 vs. ref.	r_N (real)	r_A	Pooled $\bar{S}$ (Eq. 6)	Fold S_f range	Above ref.?
DL-B: char-CNN-tiny → char-CNN	char-CNN (0.9884)	42	B0	0.9839	−0.45 pp	30.2%	4.0%	2.13× (2.39×*)	1.52–4.10×	No (ΔF1 < −0.30 pp)
DL-C: char-CNN-EE (internal exit)	char-CNN (0.9884)	42	B0	0.9874	−0.10 pp	1.1%	0.4%	3.52× (3.53×*)	3.24–3.62×	**Yes**
DL-C: char-CNN-EE (confirmation)	char-CNN (0.9862)	43	B0	0.9895	+0.33 pp	0.0%	0.005%	3.62×	3.62–3.62×	**Yes (2/2 seeds)**
Control: char-CNN-k3 (stand-alone)	char-CNN (0.9884)	42	—	0.9748	−1.36 pp	0.0%	0.0%	3.62×	3.62–3.62×	No (ΔF1 < −0.30 pp)
Stand-alone char-CNN vs. DistilBERT	DistilBERT (0.9914)	42	—	0.9884	−0.303 pp	0.0%	0.0%	80.2×	80.2–80.2×	No (0.003 pp short)
DL-A: char-CNN → DistilBERT	DistilBERT (0.9914)	42	B0	0.9911	−0.03 pp	7.1%	0.7%	**12.3**× (51.6×*)	2.95–80.2×	**Yes (seed 42)**
DL-A: char-CNN → DistilBERT (confirm.)	DistilBERT (0.9947)	43	B0	0.9914	−0.32 pp	8.9%	0.6%	**10.6**× (38.3×*)	3.93–80.2×	No (ΔF1 = −0.32 pp)
DL-A + class-wise τ (exploratory)	DistilBERT (0.9914)	42	G2	0.9920	+0.06 pp	0.7%	0.7%	51.1× (57.5×*)	32.5–80.2×	Secondary only
DL-A + class-wise τ (exploratory)	DistilBERT (0.9947)	43	G2	0.9920	−0.27 pp	1.1%	0.6%	42.5× (50.6×*)	13.7–80.2×	Secondary only

Notes: Values with * are arithmetic means of fold speedups shown for comparison. Same-run p50 latencies: for DL-B/DL-C run, char-CNN-tiny t_1 = 0.0606 ms, char-CNN t_2 = 0.3451 ms, DL-C front t_{front} = 0.0954 ms, rest t_{rest} = 0.2548 ms; for DL-A run, char-CNN t_1 = 0.563 ms (encoding 0.039 + forward 0.524), DistilBERT t_R = 45.15 ms (tokenization 0.227 + forward 44.92).

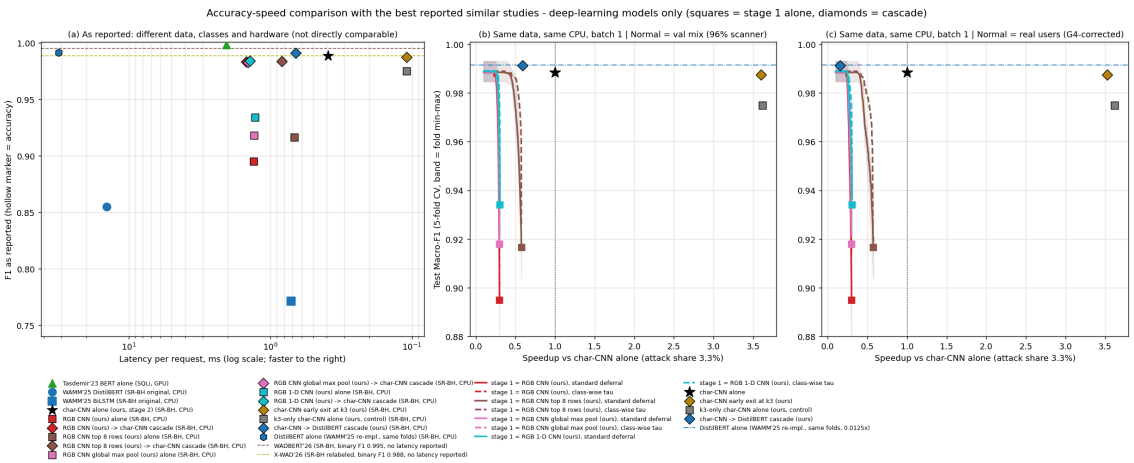


Fig. 2. Accuracy–latency trade-off of deep-learning configurations on SR-BH 2020 (5-fold CV, CPU batch 1, 1 thread). Panel (a) shows end-to-end latency alongside reference points reported by prior work under different hardware/class setups; panels (b) and (c) compare speedup vs. Macro-F1 on the exact same folds under validation benign mix and real-user benign traffic.

1) DL-B (초경량 문자 모델 → char-CNN): 1차 용량 부족에 따른 손실. 임베딩과 채널 폭을 절반으로 줄인 char-CNN-tiny는 추론 지연이 0.0606ms로 char-CNN(0.3451ms)의 1/5.7에 불과했으나, 단독 Macro-F1이 0.9601로 떨어졌다. 그 결과 실사용자 정상 요청의 30.2%를 넘기며 평균 2.13배(폴드 산술평균 2.39배) 가속을 얻었지만, 교차검증 Macro-F1이 0.9839로 char-CNN 단독 대비

−0.45%p 하락해 사전 허용 기준(0.30%p 손실 이내)을 넘지 못했다.

2) DL-C (char-CNN 내부 조기 종료): 넘김이 아닌 결합 학습 기반 모델 압축 효과. 커널 3 분기에 보조 출력 헤드를 달아 결합 학습한 char-CNN-EE는 seed 42에서 실사용자 기준 **3.52**배(F1 0.9874, 단독 대비 −0.10%p), 확인용 seed 43에서 **3.62**배(F1 0.9895, 단독 대비 +0.33%p)를 기록해 두 seed 모두에서 사전 기준을 통과하였다. 그러나 폴드별 운영점을 뜯어보면 seed 42의 5개 폴드 중 3개, seed 43의 5개 폴드 전체에서 실제 넘김률이 **0%**였다(보조 헤드 단독 Macro-F1이 이미 0.9849). 즉 이득은 "어려운 요청을 골라 커널 5·7로 넘기는 판단"에서 나온 것이 아니라, 보조 헤드 하나만으로 전체 요청을 다 처리해도 **char-CNN** 전체와 거의 같은 정확도가 나온다는 데서 비롯되었다. 이 결과가 단순히 "원래 커널 3 하나면 충분했던 것"인지 확인하기 위해 동일 크기의 단일 분기 모델(char-CNN-k3)을 처음부터 단독 학습한 대조 실험(E9-1)에서는 단독 학습 모델의 Macro-F1이 **0.9748(−1.36%p)**로 크게 떨어져 기준을 충족하지 못했다(5개 폴드 중 4개가 에폭 2~9에서 조기 종료되며 검증 점수가 최대 5%p 요동침). 반면 주 헤드(커널 3·5·7)와 결합 손실로 함께 학습한 DL-C의 보조 헤드는 에폭 12~19까지 안정적으로 수렴해 +1.01%p 더 높은 성능을 냈다(다만 폴드 5개 기준 보정 p = 0.34). 따라서 DL-C는 넘김 판단의 개선이 아니라, 다중 커널 주 헤드와의 결합 학습을 통해 얕은 단일 분기 출구를 안정적으로 압축한 효과**(2**차 모델 압축**)**로 해석하는 것이 정확하다.

3) DL-A (char-CNN → DistilBERT): 속도 개선과 정확도–속도 교환의 양면성. 동일 실행에서 측정한 종단간 지연은 char-CNN이 0.563ms, 재구현 DistilBERT가 45.15ms로 **80.2**배 차이가 났다. 즉 DistilBERT로 넘어가는 요청 1건이 char-CNN 80건을 처리하는 비용과 맞먹는다. - 속도 개선**(Speedup)**: char-CNN을 1차로 두면 단독 정확도(0.9884)가 높기 때문에 B0 게이트 기준 공격 요청의 **0.7%**, 실사용자 정상 요청의 **7.1%**(seed 42; seed 43은 8.9%)만 DistilBERT로 넘어간다(배포 기대 넘김률 $E_\pi = 6.9\%$). 덕분에 재구현 DistilBERT 단독(45.15ms) 대비 실사용자 평균 비용 배수(식 6)는 **seed 42**에서 **12.3**배**(**평균 지연 **3.67ms), seed 43**에서 **10.6**배**(**평균 지연 **4.28ms)**에 달했다(검증 분포 정상 기준으로는 각각 47.5배와 49.0배). 두 seed 모두 다섯 폴드 전체에서 1배를 넘겨(최저 폴드 배수 seed 42 2.95배, seed 43 3.93배) 일관된 10배 이상의 추론 가속을 달성하였다. - 정확도–속도 교환**(Accuracy–Speed Trade-off)**: 정확도 측면에서 이 캐스케이드는 양 끝단 단독 모델 사이의 명확한 교환 관계를 보여 준다. 먼저 가장 빠른 끝단인 **char-CNN** 단독은 DistilBERT보다 80.2배 빠르지만, Macro-F1이 seed 42에서 −0.303%p(0.9884 대 0.9914), seed 43에서 −0.85%p(0.9862 대 0.9947) 낮아 사전 허용 기준(−0.30%p)을 간발의 차(seed 42에서 0.003%p)로 통과하지 못했다. 반대로 **char-CNN → DistilBERT** 캐스케이드는 char-CNN 단독 대비 평균 지연을 6.5배(seed 42)~7.6배(seed 43) 더 쓰는 대가로 Macro-F1을 +0.28%p(seed 42, 0.9911) 및 +0.53%p(seed 43, 0.9914) 끌어올렸다(단, char-CNN 단독과의 보정 t-검정 p는 각각 0.34와 0.23으로 통계적 유의성은 미확인). - 분할 **seed**에 따른 판정 경계: 기준 모델인 재구현 DistilBERT 단독과 비교하면, **seed 42**에서는 캐스케이드가 손실 **0.03%p(0.9911** 대 **0.9914)**와 **12.3**배 가속으로 사전 고정된 세 조건을 모두 충족하여 선행 연구 재구현 모델을 정확도–속도 평면에서 지배하였다. 그러나 사전 등록한 확인 실험**(seed 43, E11)**에서는 속도 조건**(10.6**배**,** 전 폴드 **> 1**배**)**은 충족했으나 **Macro-F1** 손실이 **−0.32%p(0.9914** 대 **0.9947, 5**개 폴드 모두 음수**,** 보정 **p = 0.038)**로 허용선인 **0.30%p**를 **0.02%p** 넘어 재현되지 않았다. 원인을 분석하면 캐스케이드의 성능이 떨어진 것이 아니었다(캐스케이드 F1은 seed 42의 0.9911에서 seed 43의 0.9914로 오히려 소폭 상승). 달라진 것은 비교 기준인 DistilBERT 단독이 3에폭 미세조정의 seed 간 편차로 인해 0.9914(seed 42, 폴드 3이 0.9811로 낮았음)에서 0.9947(seed 43, 전 폴드 0.993 이상)로 크게 올라간 점이다. 전체 트래픽의 약 93%는 2차를 거치지 않고 1차 char-CNN의 판정으로 종료되므로, 2차 단독 모델의 성능이 크게 뛰더라도 1차가 확신하며 틀린 요청들은 넘김이 일어나지 않아 캐스케이드의 상한이 묶이게 된다. - 클래스별 임계값**(G2)** 결합 효과**(**탐색·병기**)**: 이미지 1차에서는 $t_2/t_1 < 1$이어서 게이트를 어떻게 고쳐도 1배를 넘을 수 없었으나, $t_R/t_1 = 80.2$인 DL-A에서는 넘김 비율 **1%p**가 곧 **char-CNN 0.8**회 분량의 지연을 좌우한다. 이 구조에 클래스별 임계값(G2)을 적용하면 실사용자 정상 요청의 과잉 넘김이 억제되어(seed 42에서 E_π가 6.87%→0.71%로 감소, 가장 느렸던 폴드 4의 배수가

2.95배→32.5배로 상승), 실사용자 평균 비용 배수가 **seed 42**에서 **12.3**배 → **51.1**배**(F1 0.9920, DistilBERT** 대비 **+0.06%p)**, **seed 43**에서 **10.6**배 → **42.5**배**(F1 0.9920, DistilBERT** 대비 **−0.27%p)**로 크게 개선되었다. 비록 G2는 E10·E11에서 보조 병기 지표로 두었으므로 기본 판정(B0 기준 seed 43 미재현)을 뒤집는 데 쓰지 않지만, 1·2차 지연 격차가 큰 딥러닝 캐스케이드에서 클래스별 임계값 조정이 폴드 간 최악 지연(tail fold latency)을 방어하는 유망한 수단임을 보여 준다.

5.6 CmdI 오탐 분석: 왜 DistilBERT를 2차로 결합하는가(사후 분석)

5.5절에서 보았듯 DistilBERT는 char-CNN보다 80배 무겁다. 그렇다면 그렇게 큰 연산 비용을 치르면서까지 DistilBERT를 2차로 두었을 때 실제로 교정되는 오류는 무엇인가? 이를 규명하기 위해 본 연구의 전 실험 과정을 관통하는 클래스별 병목과 두 모델의 5-fold 합산 예측(226,048건) 오류 교차표를 분석하였다(Table 7; 이 절은 E11 판정 이후 수행한 사후 분석이다).

1) 전 실험의 공통 병목 = 명령 삽입**(CmdI)**과 공격 종류 간 혼동**.** 본 연구에서 시험한 모든 모델과 입력 조합에서 가장 낮은 성능을 낸 클래스는 예외 없이 **CommandInjection(CmdI)**이었다. 정상·SQLi·CodeInjection 세 클래스는 대부분의 모델에서 F1 0.95~0.99 이상에 도달한 반면, CmdI F1은 고전 선형 모델(TF-IDF+LR)에서 **0.785**, 1차 RGB CNN에서 **0.756**, 경량 텍스트 신경망인 char-CNN에서도 **0.9682(seed 42) / 0.9618(seed 43)**로 다른 세 클래스(0.991~0.998)보다 뚜렷이 낮았다. 여기에는 세 가지 이유가 겹쳐 있다. 첫째, CmdI는 전체 데이터의 2.1%(학습 풀 3,886건)에 불과한 최소 클래스여서, Macro-F1 계산 시 **CmdI** 오답 **1**건이 전체 지표에 미치는 영향이 **SQLi**나 **Normal** 오답 **1**건의 약 **6**배에 달한다. 둘째, 5.7절의 템플릿 묶음 분석에서 드러나듯 CmdI는 폴드 간 요청 템플릿 공유 비율이 14.7%로 타 클래스(60~71%)보다 훨씬 낮아 표면 문자열 암기로 맞히기 어렵다. 셋째, 구문적으로 SQLi와 CmdI는 세미콜론(;), 파이프(|), 따옴표(') 주석 기호 등 명령어를 이어 붙이는 국소 구분자를 공유한다. 이 때문에 초기 캐스케이드(T1)에서도 1차 모델의 클래스별 넘김률이 CodeInjection은 13.7%인 데 비해 CmdI는 **83.7%**까지 치솟았다.

Table 7. Pooled 5-fold error decomposition comparing stand-alone char-CNN and stand-alone DistilBERT across 226,048 requests (post-hoc analysis)

Error category / metric	Seed 42: char-CNN	Seed 42: DistilBERT	Seed 42 change	Seed 43: char-CNN	Seed 43: DistilBERT	Seed 43 change
Missed attack (Attack → Normal)	791	736	−7.0%	839	777	−7.4%
Benign false alarm (Normal → Attack)	206	245	+18.9%	140	118	−15.7%
Real-user benign FPR (n = 550)	2.00%	1.82%	−0.18 pp	1.27%	1.09%	−0.18 pp
Scanner benign FPR (n = 54,600)	0.35%	0.43%	+0.08 pp	0.24%	0.21%	−0.03 pp
Attack-type confusion (blocked, wrong class)	**266**	**167**	**−37.2%**	**359**	**59**	**−83.6%**
└ SQLi misclassified as CmdI (CmdI FP)	**192**	**99**	**−48.4%**	**252**	**2**	**−99.2%**
CommandInjection (CmdI) F1	**0.9682**	**0.9784**	**+1.02 pp**	**0.9618**	**0.9903**	**+2.85 pp**
Disagreement (both / only char-CNN / only DistilBERT wrong)	726	537	422	777	561	177

2) DistilBERT가 실제로 고치는 것**:** 차단 실패가 아니라 **SQLi → CmdI** 오탐 제거**.** Table 7은 흔히 기대하는 "대형 트랜스포머가 공격을 더 잘 막는다(차단율을 높인다)"는 통념이 이 데이터에서는 성립하지 않음을 보여 준다. 공격을 정상으로 놓친 건수(차단 실패)는 char-CNN에서 DistilBERT로 가도 두 seed 모두 약 7%(791→736건, 839→777건) 줄어드는 데 그쳤고, 두 모델이 공통으로 놓친 요청(726건·777건)이 대다수였다. 정상 요청을 공격으로 잘못 막은 오탐(false alarm) 역시 seed 42에서는 오히려 증가(+18.9%)하고 seed 43에서는 감소(−15.7%)해 방향이 일관되지 않았으며, 실사용자 정상 층(550건)의 오탐률 차이는 두 seed 모두 0.18%p(요청 단 1건 차이)였다.

대신 DistilBERT의 압도적인 개선은 공격 종류 간 혼동**(attack-type confusion)**, 그중에서도 **SQLi** 요청을 **CmdI**로 잘못 분류하던 오탐**(SQLi → CmdI)**에서 나타났다. - char-CNN 단독 모델에서는 공격 종류 간 혼동 266건(seed 42) 및 359건(seed 43) 가운데 **70%** 이상**(192**건**, 252**건**)**이 **SQLi**를 **CmdI**로 오분류한 건이었다. 소수 클래스인 CmdI 입장에서는 이 오탐들이 정밀도(precision)를 직접 깎아내렸다. - DistilBERT는 이 **SQLi → CmdI** 오탐을 **seed 42**에서 **192**건 **→ 99**건**(−48.4%), seed 43**에서 **252**건 **→ 2**건**(−99.2%)**으로 줄였다**.** 그 결과 전체 공격 종류 혼동이 37.2% 및 83.6% 감소했고, CmdI 클래스 F1이 **0.9682 → 0.9784(+1.02%p, seed 42)**, **0.9618 → 0.9903(+2.85%p, seed 43)**로 도약하였다. 나머지 세 클래스의 F1 변화폭이 0.1%p 안팎이었던 점을 감안하면, 두 모델 간 전체 Macro-F1 격차는 사실상 이 SQLi → CmdI 혼동 억제에서 나온 것이다.

이러한 차이가 발생하는 구조적 이유(가설)는 수용 영역(receptive field)과 토큰 표현의 차이에 있다. char-CNN은 3·5·7바이트 길이의 1차원 합성곱 후 시간축 최대 풀링을 취하므로, 요청 어딘가에 ;, |, ' OR, — 같은 짧은 기호 연쇄가 등장하면 주변 단어와 무관하게 강한 활성값을 낸다. 반면 DistilBERT는 WordPiece 서브워드 단위로 문자열을 쪼개고 최대 512 토큰 전체에 걸쳐 자기어텐션(self-attention)을 계산하므로, 동일한 구분자 뒤에 오는 토큰이 SELECT·UNION 같은 SQL 절인지 cat·wget·/bin/sh 같은 운영체제 명령어인지를 전역 문맥에서 가려낼 수 있다.

3) 캐스케이드에서의 시사점**:** 확신도 분포와 회수 가능 영역**.** 그렇다면 1차 char-CNN이 틀리고 2차 DistilBERT만 맞힌 요청들(seed 42에서 537건, seed 43에서 561건)을 캐스케이드가 얼마나 2차로 넘길 수 있는가? 전체 226,048개 요청에서 char-CNN의 확신도가 0.99 이상인 비율은 **97.0%**에 달하지만, **DistilBERT**가 바로잡은 오답 집합**(only char-CNN wrong)**에서 **char-CNN** 확신도의 중앙값은 **0.775(seed 42)** 및 **0.766(seed 43)**으로 훨씬 낮았고, 확신도가 0.99 이상으로 과신하며 틀린 비율은 **15.1%(seed 42)**와 **10.7%(seed 43)**에 그쳤다. 이는 두 가지 사실을 동시에 설명한다. 첫째, char-CNN이 SQLi와 CmdI 사이에서 흔들릴 때는 대체로 확신도가 낮아지므로(중앙값 ~0.77), 약 7~9%의 요청만 넘기는 캐스케이드로도 상당수의 공격 종류 혼동을 DistilBERT로 보내 교정할 수 있다 (캐스케이드가 char-CNN 단독 대비 Macro-F1을 +0.28~+0.53%p 회복한 원천). 둘째, 그럼에도 약 11~15%의 교정 가능 오답에서는 char-CNN이 0.99 이상의 확신도로 틀리기 때문에 단일 임계값 게이트(B0)로는 끝내 넘기지 못하며, 이것이 seed 43처럼 DistilBERT 단독이 SQLi → CmdI 오탐을 2건까지 줄였을 때 캐스케이드가 마지막 0.32%p 격차를 좁히지 못한 구조적 한계가 된다.

5.7 견고성 점검

템플릿 묶음 교차검증**(T5).** 웹 로그 데이터셋에서는 동일한 공격 도구 패턴에서 숫자나 세션 토큰만 바뀐 유사 요청들이 학습과 시험 폴드에 나뉘어 들어가 성능을 부풀릴 위험이 있다. 실제로 요청 내 숫자열, 8자 이상 16진수, 16자 이상 긴 토큰을 마스킹한 골격 템플릿을 기준으로 보면, 기본 층화 분할의 시험 폴드 요청 중 66.5%가 학습 폴드에 동일 템플릿을 두고 있었다(단, CmdI는 14.7%로 가장 낮음). 같은 템플릿이 절대 학습과 시험에 동시에 들어가지 않도록 묶음 분할(StratifiedGroupKFold)을 수행한 결과, RGB CNN은 −0.06%p(0.8949→0.8943), char-CNN은 +0.10%p(0.9884→0.9894)로 사전 판정 기준(2.0%p 이상 하락 시 부풀림 판정)에 해당하지 않았다.

분할 및 초기화 **seed** 산포**(T10).** 교차검증 분할과 가중치 초기화를 바꾼 4개 seed(42~45)에서 단독 모델의 평균 Macro-F1은 RGB CNN이 0.8949~0.9229(평균 0.9078, 표준편차 1.15%p), char-CNN이 0.9862~0.9901(표준편차 0.18%p)이었다. 기본 판정에 쓴 seed 42는 네 seed 중 RGB CNN 점수가 가장 낮은 분할이었으므로, 이미지 모델에서 단일 seed의 1~2%p 차이는 분할 잡음 범위 안에 있다. 5.5절에서 DL-A가 seed 42(−0.03%p)와 seed 43(−0.32%p) 사이에서 판정이 갈린 것 또한 미세조정 트랜스포머의 seed 간 변동(0.9914→0.9947, +0.33%p)이 허용폭(0.30%p)과 맞먹는 크기이기 때문이다.

봉인 시험 집합 사용 감사**(T11).** 본 연구 전 과정에서 고정 시험 집합(39,891행)은 총 9회 참조되었고, 그 결과를 보고 설계를 변경한 사례는 5.2절의 1차 모델 학습 방식(언더샘플링 → 클래스 가중치) 1건뿐이었다. 해당 결정은 봉인된 학습+검증 풀의 5-fold 교차검증에서 독립적으로 재검증하여 시험 집합 과적합이 아님을 확인하였다.

VI. 논의

캐스케이드가 실질적 이득을 내는 두 조건. 본 연구의 일련의 음성 및 양성 결과는 웹 공격 탐지 캐스케이드가 성립하기 위한 물리적·통계적 조건을 명확히 가른다. 첫째, 전처리를 포함한 1차 지연 t_1이 2차 지연 t_2보다 충분히 작아야 한다($t_1 \ll t_2$). 이미지 CNN 1차는 48×48 RGB 변환 전처리(0.407ms) 만으로도 이미 실제 길이를 읽는 char-CNN 전체(0.393ms)보다 느려 이 조건을 원천적으로 위배하였다. 둘째, 1차 모델이 대다수 정상·공격 요청을 높은 확신도로 맞혀 배포 넘김률 E_π를 수 퍼센트 이내로 묶어 두어야 한다. 딥러닝 후보 중 두 조건을 모두 채운 것은 char-CNN → DistilBERT(t_R/t_1 = 80.2, $E_\pi \approx 7\%$)뿐이었고, 이때 평균 비용 기준 10.6~12.3배의 가속이 실현되었다. 반면 2차 모델이 이미 매우 빠를 때(char-CNN 0.35ms)는 두 모델로 나누는 캐스케이드(DL-B)보다 한 모델 내부에서 결합 학습된 얕은 분기로 조기 종료하는 구조(DL-C, 3.5~3.6배)가 손실 없이 작동하였다.

넘김 판단의 한계와 공격 종류 오탐의 결합. 1차 확률 변환(G1), 클래스별 임계값(G2), 내부 표현 라우터(G3), 특징 공간 거리 점수(G6)가 모두 사전 채택 기준을 넘지 못한 사실은 Jitkrittum 등[6]이 밝힌 확신도 기반 넘김의 이론적 한계를 실트래픽 웹 보안 데이터에서 실증한다. 특히 5.6절의 오류 분해는 왜 트랜스포머 2차가 필요한지와 왜 확신도 게이트가 완벽히 수렴하지 못하는지를 동시에 보여 준다. DistilBERT의 실질적 기여는 공격 차단율 상승이 아니라 char-CNN의 고질적 약점인 **SQLi → CmdI** 공격 종류 오탐을 절반 이하**(seed 43**에서는 **252→2**건**)**로 제거하는 데 있었다. 이러한 혼동 요청의 상당수(85~89%)는 char-CNN 확신도가 0.99 미만(중앙값 ~0.77)이라 캐스케이드가 2차로 넘겨 구제할 수 있었지만, 나머지 11~15%는 char-CNN이 0.99 이상으로 과신하며 틀린 경우여서 어떠한 1차 확신도 임계값으로도 복구할 수 없었다.

실무 배포 관점의 선택 지침. 인라인 WAF 운영자에게 본 논문의 결과는 구체적인 설계 기준을 제시한다. 1. 이진 차단**(**정상 대 공격**)**이나 약 **0.3~0.8%p**의 **Macro-F1** 차이가 허용되는 환경이라면, 캐스케이드 없이 **char-CNN** 단독**(**또는 결합 학습된 **DL-C** 조기 종료 출구**)**을 배치하는 것이 DistilBERT보다 80배(DL-C는 약 280배) 빠르면서 차단 실패율도 거의 차이가 없다. 2. 관제·대응 자동화를 위해 **SQLi**와 **CmdI** 등 세밀한 공격 유형 분류 정확도**(**특히 소수 클래스인 **CmdI** 정밀도**)**가 필수적인 환경이라면 DistilBERT가 유의미한 이득을 주며, 이때 char-CNN → DistilBERT 캐스케이드는 DistilBERT 단독 연산 비용을 약 1/11~1/12(클래스별 임계값 G2 적용 시 탐색 기준 1/40~1/50)로 줄이는 실용적 절충안이 된다.

한계 및 향후 과제. 첫째, 모든 지연은 단일 데스크톱 CPU의 1 스레드(배치 1)에서 파이썬·PyTorch 구현 그대로 측정한 값이다. 멀티스레드나 GPU 배치 추론에서는 트랜스포머의 병렬화 효율이 높아져 t_R/t_1 비율이 80배보다 줄어들 수 있다. 둘째, 교차검증 폴 내 실사용자 정상 요청이 550건(폴드당 약 110건)뿐이어서 실사용자 r_N 추정치의 폴드 간 분산이 크다(DL-A에서 특정 폴드의 배수가 2.95~3.93배로 낮아진 원인). 셋째, DistilBERT 재구현은 원 논문(WAMM[5])의 128 토큰 대신 잘림이 없는 512 토큰과 Adam 최적화기를 사용하였다; 원 설정인 128 토큰 판본에서 CmdI 문맥 잘림(26.5%)이 실제 성능과 캐스케이드 배수에 미치는 영향은 별도 후속 검증(E12)으로 다루고자 한다. 넷째, 5.3절의 꼬리 오답 라벨 감사(70.9% 불일치)는 대형 언어모델 1인 판정이므로, 향후 보안 전문가 2인 독립 감사를 통한 판정자 간 일치도(Cohen's κ) 확보가 필요하다. 다섯째, 캐스케이드는 공격자가 의도적으로 1차 확신도를 낮추어 모든 요청을 DistilBERT로 넘기게 만드는 비용 기반 공격(slowdown attack)[29]에 노출된다. 실제로 본 트랙에서 공격 페이로드에 URL 인코딩 1회를 적용하자 공격 넘김률이 약 1.8배 증가하였으며, 특히 DL-A에서는 넘긴 요청 1건이 80배의 비용을 유발하므로 변형 공격 하에서의 게이트 방어와 적응형 공격 평가는 중요한 후속 과제이다.

VII. 결론

본 논문은 웹 요청 바이트 이미지 CNN이 문자 수준 CNN(char-CNN)보다 유의하게 부정확하다는 음성 결과에서 출발하여, 웹 공격 탐지 캐스케이드의 속도–정확도 교환과 넘김 판단의 한계를 실트래픽 기반 SR-BH 2020에서 재평가하였다. 전처리를 포함하고 패딩을 제거한 공정 지연 기준으로 재면 이미지 1차 캐스케이드의 속도 이득은 1.22배에서 0.27배로 사라졌고, 확신도 신호 교체·클래스별 임계값·학습형 라우터·특징 공간 거리 점수 중 어느 것도 과잉 넘김을 사전 기준만큼 줄이지 못했다.

또한 공격 비중이 높은 시험 집합의 넘김률만으로 게이트를 고르면 배포 공격 비율(3.3%)과 실사용자 정상 트래픽에서는 오히려 더 느린 게이트를 택하게 됨을 보였다.

전처리 병목을 없앤 딥러닝 구성에서는 두 가지 대안이 서로 다른 방식으로 유효했다. char-CNN 내부의 커널 3 분기에 보조 출구를 결합 학습한 조기 종료 모델(DL-C)은 단독 모델과 같은 정확도를 3.5~3.6배 빠르게 냈으나 그 원천은 넘김 판단이 아니라 얕은 출구의 모델 압축 효과였다. 한편 char-CNN(1차)과 DistilBERT(2차)를 결합한 캐스케이드(DL-A)는 char-CNN의 가장 큰 약점인 SQLi → CmdI 공격 종류 오탐을 대폭 줄이는 DistilBERT의 장점을 취하면서 실사용자 기준 평균 지연을 10.6~12.3배 단축하였으나, 엄격한 사전 정확도 허용 기준(손실 0.30%p 이내)은 seed 42(−0.03%p)에서만 충족하고 확인용 seed 43(−0.32%p)에서는 경계를 넘어 분할 seed에 따른 재현 한계를 드러냈다. 결국 캐스케이드 탐지기의 성능 주장은 종단간 지연 정의, 배포 공격 기저율, 실사용자 정상 트래픽의 넘김률, 그리고 다중 seed에서의 분산을 함께 보고할 때에만 성립한다.

References

[1] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," Proceedings of the 31st USENIX Security Symposium, pp. 3971-3988, Aug. 2022.

[2] L. Nataraj, S. Karthikeyan, G. Jacob, and B.S. Manjunath, "Malware images: visualization and automatic classification," Proceedings of the 8th International Symposium on Visualization for Cyber Security (VizSec), pp. 1-7, Jul. 2011.

[3] F. Sureda Riera, J. Bermejo Higuera, J.R. Bermejo Higuera, J.J. Martínez Herraiz, and J.A. Sicilia Montalvo, "A new multi-label dataset for web attacks CAPEC classification using machine learning techniques," Computers & Security, 120, 102788, Sep. 2022.

[4] K. Tasdemir, R. Khan, F. Siddiqui, S. Sezer, F. Kurugollu, S.B. Yengec-Tasdemir, and A. Bolat, "Advancing SQL injection detection for high-speed data centers: a novel approach using cascaded NLP," arXiv:2312.13041, Dec. 2023.

[5] H. Osama, O. Elebiary, Y. Qassim, M. Amgad, A. Maghawry, A. Saafan, and H. Ghalwash, "Enhanced web payload classification using WAMM: an AI-based framework for dataset refinement and model evaluation," arXiv:2512.23610, Dec. 2025.

[6] W. Jitkrittum, N. Gupta, A.K. Menon, H. Narasimhan, A.S. Rawat, and S. Kumar, "When does confidence-based cascade deferral suffice?," Advances in Neural Information Processing Systems (NeurIPS), Dec. 2023.

[7] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, and H. Jin, "VulCNN: an image-inspired scalable vulnerability detection system," Proceedings of the 44th International Conference on Software Engineering (ICSE), pp. 2365-2376, May 2022.

[8] E. Eroğlu Demirkan and M. Aydos, "Enhancing malware detection via RGB assembly visualization and hybrid deep learning models," Applied Sciences, 15(13), 7163, Jun. 2025.

[9] J.R. Tadhani, V. Vekariya, V. Sorathiya, S. Alshathri, and W. El-Shafai, "Securing web applications against XSS and SQLi attacks using a novel deep learning approach," Scientific Reports, 14, 1803, Jan. 2024.

[10] K. Luo, Y. Xie, S. Zhao, and J. Pan, "WADBERT: dual-channel web attack detection based on BERT models," arXiv:2601.21893, Jan. 2026.

[11] M. Bitussi and R. Doriguzzi-Corin, "X-WAD: explainable web anomaly detection," arXiv:2608.27172, Aug. 2026.

[12] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, Oct. 2019.

[13] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," Advances in Neural Information Processing Systems (NIPS), pp. 649-657, Dec. 2015.

[14] X. Wang, D. Kondratyuk, E. Christiansen, K.M. Kitani, Y. Movshovitz-Attias, and E. Eban, "Wisdom of committees: an overlooked approach to faster and more accurate models," International Conference on Learning Representations (ICLR), Apr. 2022.

[15] M. Sevri and H. Karacan, "Two stage deep learning based stacked ensemble model for web application security," KSII Transactions on Internet and Information Systems, 16(2), pp. 632-657, Feb. 2022.

[16] A. Kutlimuratov, F. Rakhmatov, J. Khamzaev, I. Saymanov, P. Allayarov, G. Bekbaev, S. Otamurodov, and F. Makhmudov, "A lightweight cascade-based framework for real-time zero-day attack detection," Computers, 15(3), 174, Mar. 2026.

[17] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," International Conference on Learning Representations (ICLR), Apr. 2017.

[18] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," Advances in Neural Information Processing Systems (NIPS), pp. 4878-4887, Dec. 2017.

[19] D. Madras, T. Pitassi, and R. Zemel, "Predict responsibly: improving fairness and accuracy by learning to defer," Advances in Neural Information Processing Systems (NeurIPS), pp. 6147-6157, Dec. 2018.

[20] H. Mozannar and D. Sontag, "Consistent estimators for learning to defer to an expert," Proceedings of the 37th International Conference on Machine Learning (ICML), pp. 7076-7087, Jul. 2020.

[21] A. Kag, I. Fedorov, A. Gangrade, P. Whatmough, and V. Saligrama, "Efficient edge inference by selective query," International Conference on Learning Representations (ICLR), May 2023.

[22] S. Enomoto and T. Eda, "Learning to cascade: confidence calibration for improving the accuracy and computational cost of cascade inference systems," Proceedings of the AAAI Conference on Artificial Intelligence, 35(8), pp. 7331-7339, Feb. 2021.

[23] S. Teerapittayanon, B. McDanel, and H.T. Kung, "BranchyNet: fast inference via early exiting from deep neural networks," Proceedings of the 23rd International Conference on Pattern Recognition (ICPR), pp. 2464-2469, Dec. 2016.

[24] K. Lee, K. Lee, H. Lee, and J. Shin, "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," Advances in Neural Information Processing Systems (NeurIPS), Dec. 2018.

[25] J. Ren, S. Fort, J. Liu, A.G. Roy, S. Padhy, and B. Lakshminarayanan, "A simple fix to Mahalanobis distance for improving near-OOD detection," arXiv:2106.09022, Jun. 2021.

[26] Y. Sun, Y. Ming, X. Zhu, and Y. Li, "Out-of-distribution detection with deep nearest neighbors," Proceedings of the 39th International Conference on Machine Learning (ICML), Jul. 2022.

[27] H. Jiang, B. Kim, M.Y. Guan, and M. Gupta, "To trust or not to trust a classifier," Advances in Neural Information Processing Systems (NeurIPS), Dec. 2018.

[28] P.F. Jaeger, C.T. Lüth, L. Klein, and T.J. Bungert, "A call to reflect on evaluation practices for failure detection in image classification," International Conference on Learning Representations (ICLR), May 2023.

[29] S. Hong, Y. Kaya, I.-V. Modoranu, and T. Dumitraş, "A panda? No, it's a sloth: slowdown attacks on adaptive multi-exit neural network inference," International Conference on Learning Representations (ICLR), May 2021.

[30] V.J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J.S. Gardner, I. Hubara, S. Idgunji, T.B. Jablin, J. Jiao, T.S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A.T.R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, "MLPerf inference benchmark," Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA), pp. 446-459, May 2020.

[31] S. Axelsson, "The base-rate fallacy and its implications for the difficulty of intrusion detection," Proceedings of the 6th ACM Conference on Computer and Communications Security (CCS), pp. 1-7, Nov. 1999.

[32] C. Guo, G. Pleiss, Y. Sun, and K.Q. Weinberger, "On calibration of modern neural networks," Proceedings of the 34th International Conference on Machine Learning (ICML), pp. 1321-1330, Aug. 2017.

[33] C. Nadeau and Y. Bengio, "Inference for the generalization error," Machine Learning, 52(3), pp. 239-281, Sep. 2003.

[34] L.I. Kuncheva and C.J. Whitaker, "Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy," Machine Learning, 51(2), pp. 181-207, May 2003.

[35] Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J.V. Dillon, B. Lakshminarayanan, and J. Snoek, "Can you trust your model's uncertainty? Evaluating predictive uncertainty under dataset shift," Advances in Neural Information Processing Systems (NeurIPS), Dec. 2019.

[36] G. Lucz and B. Forstner, "A thirty-day dataset of malicious HTTP requests blocked by OWASP ModSecurity on a production web server," Data, 10(11), 186, Nov. 2025.